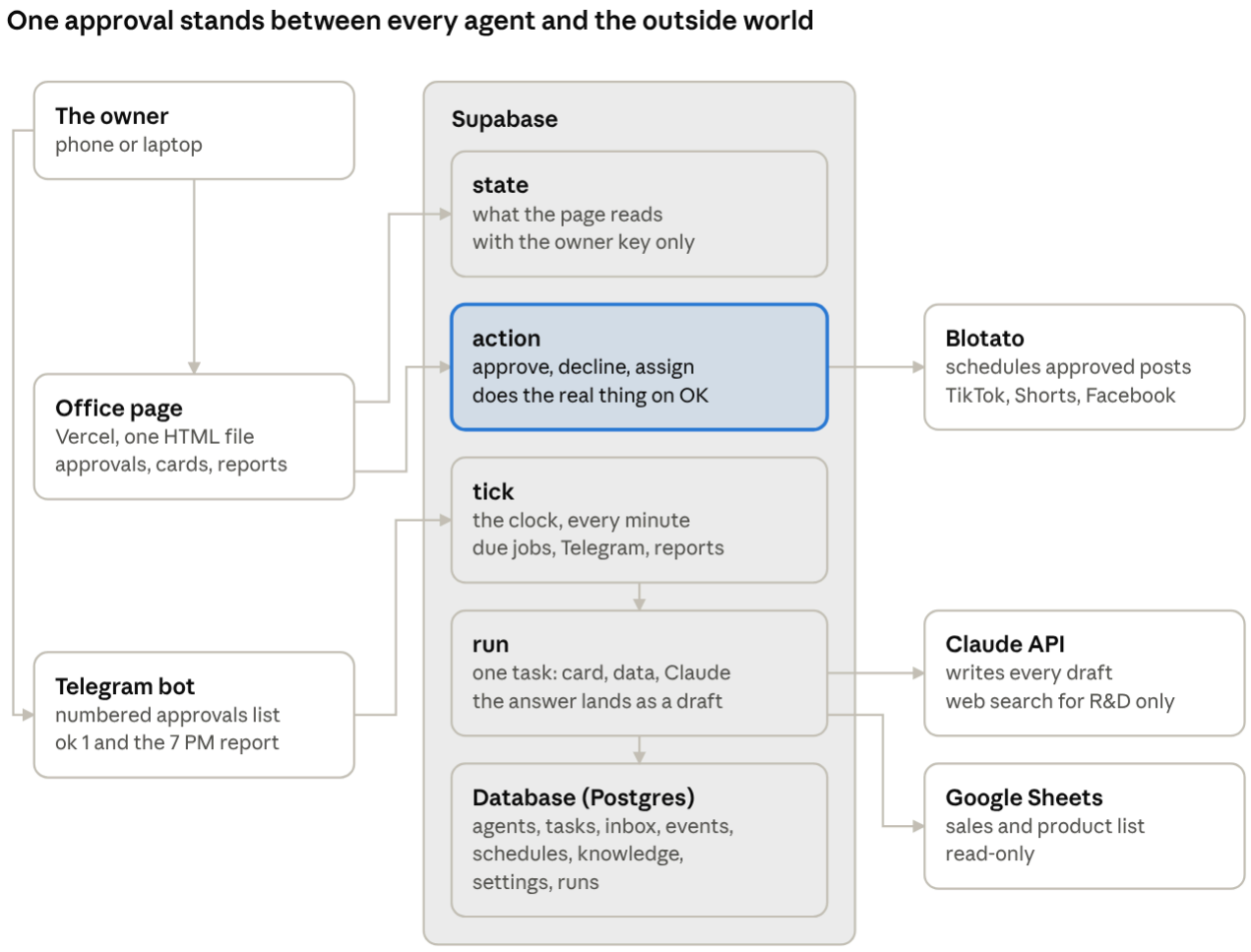
AI Office Starter Kit: Skincare Brand

Nov 13, 2026 · @Jazmine de Luna

A small AI office for a skincare brand, built on the same parts as Jazmine's Agent Office: AI agents that research products, plan and schedule social posts through Blotato, and report sales from a Google Sheet, with every outgoing action waiting for the owner's OK. Start with 7 agents in 3 teams plus a chief of staff; add more only when a job repeats.

At a glance



architecture · 2 ways in, 4 functions, 3 outside services

The owner works through the office page or Telegram; four Supabase functions do the work, and only the action function reaches Blotato, after his OK.

The team

Seven agents in three teams, plus a chief of staff who talks to the owner. Each agent is one row in the database with a written job card; the names and anime looks are his to choose.

Agent	Team	Job	Runs	Needs the owner's OK for
Chief of staff	Leadership	Collects every team's updates, sends one daily report, chases open approvals, answers him on Telegram	Every day 7:00 PM	Nothing; she only reports
Trend analyst	R&D	Finds rising skincare trends, ingredients and hooks on TikTok, Reddit and news	Every Monday 6:00 AM	Nothing; a research note
Product researcher	R&D	Writes product briefs: ingredients, claims, competitors, price points, what reviewers love and hate	When he assigns a product	Nothing; a research note
Review miner	R&D	Reads his own and competitors' reviews and comments, sorts complaints and praise by theme	Every Wednesday 6:00 AM	Nothing; a research note
Content writer	Social Media	Writes captions and short-video scripts from R&D's findings and his product list	Every day 8:00 AM	Every caption and script
Post scheduler	Social Media	Turns approved content into a posting plan and schedules it through Blotato	When he approves content	Every posting plan

Agent	Team	Job	Runs	Needs the owner's OK for
Sales tracker	Finance	Reads the sales sheet, writes the daily sales note and the month-end summary	Every day 6:00 PM; 1st of the month	Nothing; a report

Every card has the same fields: the job, where the information comes from, what the agent does alone, what needs the owner's OK, who gets the work next, a playbook of house style, and a KPI with a goal. Jazmine's office proved that a sharp card matters more than a clever prompt.

How a piece of work flows



one piece of work · 4 steps, 1 decision

Every job takes this path, whether a schedule started it or he assigned it. A "no" drops the draft; a change request goes back to the same agent once, with his note.

The database

Eight tables carry the whole office; create them first in Supabase (Postgres). Jazmine's office grew to about 40 tables, but these eight are the ones everything else hangs on.

Table	One row is	Key columns
agents	One agent	id, name, role, team, sort, connected, card (JSON: job, data, alone, needs_ok, hands_off, playbook), kpi_label, kpi_goal
tasks	One job for one agent	id, agent_id, title, prompt, source (schedule, owner, system), status (queued, working, done, failed), result_inbox_id, attempts, error
inbox	One thing for the owner	id, agent_id, kind (approval or delivery), title, body, payload (JSON, e.g. the posting plan), status (open, approved, declined), decided_at
events	One thing that happened	id, at, agent_id, type (task_started, delivered, approval_requested, posted, problem), title, task_id, inbox_id
schedules	One repeating job	id, agent_id, title, prompt, cadence (daily, weekly, monthly), at_hour, at_minute, dow, dom, enabled, next_at
knowledge	One house rule or lesson	id, kind (house, lesson), agent_id (null = everyone), title, body
settings	One switch or remembered value	key, value (JSON), updated_at
runs	One AI call	id, agent_id, task_id, model, input_tokens, output_tokens, cost_usd

Keys and tokens (Claude, Blotato, Telegram, Google) live in Supabase secrets, never in a table the page can read. The page reads through one function with the owner's key; it never touches the tables directly.

The backend functions

Four Supabase Edge Functions (Deno, TypeScript) do all the work; a database cron job pings the clock once a minute. Shared code (the Claude call, Blotato, Sheets, Telegram) sits in one shared folder all four import.

Function	Owns	Called by
tick	The clock: due schedules become tasks, queued tasks get started, Telegram is read and answered, daily jobs fire	pg_cron, every minute
run	One task: loads the agent's card and the house rules, gathers its data, asks Claude, puts the result in the inbox as an approval or a delivery	tick, and action when the owner assigns something
action	Everything the owner does: assign, approve, decline, ask for changes; on approve it does the real thing (schedules the posts)	The office page and the Telegram bot
state	What the page reads: agents, inbox, events, reports	The office page, with the owner's key

Two rules keep it safe. An agent never sends, posts or pays on its own; only the action function does, and only after an approval. And the run function writes drafts, never decisions, so a bad answer costs a draft, not a post.

Social media through Blotato

The post scheduler never posts by itself: she writes a plan, the office checks it against his real Blotato accounts, he approves it, and only then is each post scheduled. One Blotato API key, stored as a Supabase secret, covers every connected account.

1. **Read his accounts.** GET `https://backend.blotato.com/v2/users/me/accounts` lists each connected account with its id; a Facebook account's Pages, each with its page id, come from /users/me/accounts/{id}/subaccounts. Show the scheduler this list, so she can only pick real ones.
2. **The scheduler writes a plan.** Her answer ends with a fenced block of posts, one per line: platform, account id, page id for Facebook, caption, media links, time.
3. **Code checks the plan, not the AI.** Drop any post with an unknown account, a caption over the platform's limit (Instagram 2,200, Facebook 5,000), Instagram without media, or a time already past. What survives is exactly what gets sent.
4. **He approves.** The plan he reads is written by code from the checked plan, so it is what will go out.

5. **Schedule each post.** One POST to `/v2/posts` per post, with `scheduledTime` set; never publish on the spot.

```
{
  "post": {
    "accountId": "12345",
    "content": { "text": "caption", "mediaUrls": ["https://.../video.mp4"],
"platform": "tiktok" },
    "target": { "targetType": "tiktok", "privacyLevel": "PUBLIC_TO_EVERYONE",
"disabledComments": false, "disabledDuet": false, "disabledStitch": false,
"isBrandedContent": false, "isYourBrand": false, "isAiGenerated": false }
  },
  "scheduledTime": "2026-11-20T11:00:00Z"
}
```

Facebook adds `"pageId"` to the target. YouTube Shorts add `title`, `privacyStatus: "public"`, `isMadeForKids: false`. Media links must be public files Blotato can download: a Supabase Storage public link works; a Google Drive share page does not, but its direct download link does.

Rules that saved Jazmine's office: one plan per approval, never several plans merged; a reel planned for TikTok is also planned for Shorts and the Facebook Page in the same plan; no post time inside the next 10 minutes; and keep Blotato's rate limit in mind (send in small batches, retry later).

Finance from a Google Sheet or Excel

The sales tracker only reads his existing sheet; his own system stays the source of truth and the office never writes to it. Code reads the numbers and adds them up; the AI only explains them, so a total is never invented.

1. **Connect once, read-only.** He signs in with Google once (OAuth, scope `spreadsheets.readonly`, plus `drive.readonly` if the office should find the file by name). The refresh token is kept as a secret; the office gets a fresh access token each run.
2. **Read the tab.** GET `https://sheets.googleapis.com/v4/spreadsheets/{sheetId}/values/{tab}!A1:Z` returns rows. The first row names the columns; map them once in settings (date, product, quantity, amount, channel).
3. **Count in code.** Today, this week, this month, by product and by channel, against his monthly target. Store the result in settings as the day's numbers.

4. **The agent writes the note.** Every day 6:00 PM: today's sales, the best and slowest product, anything unusual (a day 50% under the 7-day average). On the 1st: the month-end summary.
5. **The chief of staff carries it.** The note goes into the 7:00 PM daily report, not as its own Telegram message.

Excel works the same way if the file lives in OneDrive (Microsoft Graph, read-only) or is saved to Google Drive as a Sheet. A CSV export he drops in a folder is the fallback: no sign-in, but the numbers are only as fresh as the last export.

If the shop runs on Shopify or a payment link (PayMongo, Stripe), read that instead and keep the sheet for what is sold offline; Jazmine's office reads Stripe and her payment sheets side by side.

R&D for skincare

R&D writes notes, never posts: its output feeds the content writer and his product decisions. Turn on Claude's web search for these three agents only; it is the one place it pays for itself.

Agent	Reads	Writes	Hands to
Trend analyst	TikTok and Shorts skincare trends, ingredient news, Reddit (r/SkincareAddiction, r/AsianBeauty, r/PHMakeupAndBeauty)	Monday brief: 5 rising trends, each with the hook, the ingredient, and a post idea for his products	Content writer
Product researcher	Competitor listings on Shopee, Lazada and brand sites, ingredient databases, his product sheet	A one-page brief per product: claim, key ingredients, price band, 3 competitors, what reviewers praise and complain about	The owner; content writer
Review miner	His own reviews and comments, top competitors' reviews	Wednesday note: complaints and praise grouped by theme, with real quotes	Product researcher; content writer

Add a house rule for claims: no medical claims (cures, treats, heals), no "FDA approved" unless the product's FDA Philippines notification is on file, and every "clinically proven" line names its study. The content writer reads the same rule, and the QA check can block a caption that breaks it.

A product list in a sheet (name, size, price, key ingredients, FDA notification number) is the one input R&D needs from him; read it like the sales sheet.

The office page and anime characters

The page is one static HTML file on Vercel that reads the state function every few seconds. Build the useful screens first (Approvals, Inbox, the agents' cards, Reports); the animated office is the fun layer on top.

Three ways to show the anime characters, cheapest first:

Option	What it is	Cost and effort	Good for
Portraits	One anime image per agent (face and bust), shown on cards, chat and approvals	1 image each; an afternoon	Start here; works on any phone
2D office	A drawn office background with each character as a sprite at a desk, a speech bubble when working	1 background + 2 to 3 poses each; a few days	The "office" feeling without 3D
3D office	Each character as a rigged 3D model walking between rooms, like Jazmine's (three.js)	About 38 Higgsfield credits per character, plus 1 to 2 weeks of page work	Later, once the agents earn their keep

To keep the anime look consistent across the cast, write one style line and reuse it in every image prompt: the same art style, line weight, palette and camera angle, a plain background, full body facing the camera. Generate the portrait first and use it as the reference for any later pose or 3D model.

Agents keep their ids; the look is just a file per id (for example `portraits/<id>-face.webp`). Changing a character's art never touches the backend.

Build order

Five phases, each usable on its own; do not start the next until the last one's "done" is true. Roughly 3 to 4 weeks of evenings with Claude Code doing the typing.

1. **Skeleton (week 1).** Supabase project, the eight tables, the four functions, the cron ping, the Claude key as a secret. Done when: he assigns a task to one agent from a bare page and its draft lands in the inbox.

2. **Approvals and Telegram (week 1).** Approve, decline and "change this" on the page; a Telegram bot paired to his chat that sends the numbered approvals list and takes `ok 1`. Done when: he approves a draft from his phone.
3. **Social media (week 2).** Blotato key, the accounts list, the content writer and the post scheduler, the plan check, scheduling on approve. Done when: a reel he approved shows as scheduled in Blotato on TikTok, Shorts and his Facebook Page.
4. **R&D and Finance (week 3).** Web search for the three R&D agents; Google sign-in, the sales sheet read and the 6:00 PM note; the 7:00 PM report. Done when: one evening report carries the day's sales and a trend idea.
5. **The office and the characters (week 4 onward).** Anime portraits on every card, then the 2D office; 3D only if he wants it. Done when: he opens the page and sees his team at work.

Cost to run, roughly: Supabase and Vercel free tiers; Claude about US$1 to US$3 a day for seven agents on daily jobs; Blotato's own plan.

Lessons from building Jazmine's office

Copy these:

- **Approvals are the product.** Every outgoing action is a draft he approves, in the office or by replying `ok` in Telegram. Nothing posts, sends or pays without it.
- **Code checks, the AI drafts.** Totals, account ids, dates, limits and duplicates are checked in code before he sees a draft.
- **Quiet by default.** Telegram carries only real problems, one numbered list of drafts to approve a few times a day, and the 7:00 PM report. Everything else waits in the office. Jazmine had to ask twice for less noise.
- **One schedule table.** Every repeating job, with its day and time, is a row; the agents page shows them per agent, so he always knows who does what and when.
- **House rules in the database.** When he says "always" or "never", it becomes a knowledge row every agent reads, not a code change.

Avoid these:

- **Two builders on one backend.** Two Claude sessions deploying the same functions overwrote each other's work for a day. One session owns the backend; others ask it.
- **Too many agents too early.** Start with seven; add one only when the same task keeps landing on him.
- **Merging approvals.** One approval should mean one plan. A picker that merged several plans scheduled reels to the wrong accounts.

- **Secrets in chat or files.** Keys go into Supabase secrets through a form, never pasted into a conversation, a sheet or the page.